

第五章 软件测试与维护 AI讲解

开篇导读

一句话概括：软件测试的目标是"发现错误"而非"证明程序正确"，黑盒测试看"外部行为"，白盒测试看"内部逻辑"，测试分层次推进，维护贯穿软件生命周期。

本章在考试中的地位：最高频重点章（用户明确指定重点）。黑盒测试方法、白盒测试覆盖率排序、 α/β 测试区分、集成测试策略、维护类型占比是必考点。本章坑点极其密集——尤其是覆盖率之间的偏序关系、条件覆盖与判定覆盖的不可比性、 α/β 测试的区分标准，是出题人最爱设陷阱的地方。

一、软件测试的目标

核心概念

软件测试的目标是发现程序中的错误，而不是"证明程序正确"。

为什么重要

理解测试目标是本章的基石。很多关于测试的错误认知都源于对目标的误解：

- 正确认知：好测试 = 发现了尚未发现的错误 = 高发现概率 + 高覆盖
- 错误认知：测试是为了"证明程序没有错误"——这是不可能的（测试只能证明"有错"，不能证明"无错"）

关键原则

1. 测试目标是发现错误
2. 好测试 = 高发现概率的测试
3. 由独立第三方测试效果更好（开发者测自己的代码会有盲区）

易混辨析

考法	易错点
"测试的目标是什么"	是"发现错误"，不是"证明程序正确"或"证明没有错误"
"谁能测试"	独立第三方测试效果更好，开发者测自己代码效果差（有心理盲区）

记忆技巧

口诀："测试找错，不证对"——测试是为了发现错误，不是为了证明程序正确。

二、黑盒测试 核心

核心概念

黑盒测试（功能测试/基于规格说明的测试）：把程序看作一个打不开的"黑盒子"，不考虑内部结构，只根据需求规格说明书，检查程序的功能是否符合要求。

依据

- 需求规格说明书（SRS）、用户手册等外部文档

适用阶段

- 系统测试、验收测试（关注功能是否符合需求）

黑盒测试的6种方法（必背）

1. 等价类划分法（最基础）

- 是什么：把所有可能的输入数据划分成若干等价类（子集），从每个等价类中取少数代表值作为测试数据。理论依据是"同一等价类中的任一输入都等价"，测一个就能代表整类。
- 有效等价类：符合需求的输入（如年龄18-60）
- 无效等价类：不符合需求的输入（如年龄<18或>60）
- 划分原则（多种情形）：
 - 规定了取值范围（如1-100）→1个有效（1-100）+2个无效（<1、>100）
 - 规定了枚举值（如"红/绿/蓝"）→每个值1个有效+1个无效（如"黄"）
 - 规定了布尔值（如"是否会员"）→1个有效（是）+1个无效（否）
 - 规定了数据集合（如"姓名"）→集合内1个有效+集合外1个无效

完整例题：注册时用户名要求6-10位字符

- 有效等价类：6-10位字符（1个）
- 无效等价类：<6位（如"abc"）、>10位（如"abcdefghijkl"）、非字符（如"abc123!"含特殊符号）（3个）

• 测试用例：取"abcdef"（有效）、"abc"（太短）、"abcdefghijkl"（太长）、"abc123!"（含特殊符号）
恍然大悟：为什么等价类能省测试用例？因为同一类里任一输入都"等价"——测一个就能代表整类，这是用"代表性"代替"穷举"，把无限可能压缩成有限几个。

2. 边界值分析法（最常用，等价类的补充）

- 是什么：在等价类的边界处取值作为测试数据的方法。
- 取值：上点（边界值本身，如10和50）、离点（边界附近外的值，如9和51）、内点（范围内的任意值，如30）
- 例：范围10-50，边界值取10、50（上点），9、51（离点），30（内点）

为什么错误集中在边界？（理论根基）

编程时最常犯的错是"差一个"——比如该用 $\geq$ 却写成 $>$ ，该用 $<$ 却写成 $\leq$ 。这种错误在范围内部不会暴露（如输入30， >10 和 ≥ 10 都为真），但在边界处会暴露（输入10， >10 为假但 ≥ 10 为真）。所以边界是"差一个"错误的"放大镜"，专门捕捉这类bug。

恍然大悟：边界值为什么能抓到等价类抓不到的错？因为等价类取"代表值"（如30），刚好避开了边界——而 $>$ vs $\geq$ 这种错误只在边界值10处才暴露。等价类管"中间"，边界值管"边缘"，两者互补才能全覆盖。

关系：边界值分析是等价类划分的补充，不是替代！两者通常结合使用——等价类找"代表值"，边界值补"边缘值"。

3. 因果图法

- 是什么：用图表示输入条件（因）和输出结果（果）之间逻辑关系的方法。当输入条件之间存在组合/约束关系时，等价类划分无法处理这种组合，因果图法可以系统地分析"什么因导致什么果"。
- 四种基本逻辑关系：
 - 恒等（ $=$ ）：因成立→果成立（如"会员=true"→"享折扣=true"）
 - 非（ $\neg$ ）：因成立→果不成立（如"黑名单=true"→"允许下单=false"）
 - 或（ $\vee$ ）：任一因成立→果成立（如"有优惠券 $\vee$ 有会员"→"享折扣"）
 - 与（ $\wedge$ ）：所有因成立→果成立（如"收入达标 $\wedge$ 信用良好"→"放贷"）
- 适用：输入条件之间存在组合/约束的情况（如"申请贷款"受"收入""信用""担保"多个条件共同约束



)

因果图→判定表的转化：因果图画完后，把所有条件的真假组合列成表格（判定表），每行对应一个测试用例。因果图是"分析工具"，判定表是"输出工具"。

4. 判定表驱动法

- 是什么：用表格列出所有条件组合及其对应动作的方法。判定表由四部分组成：条件桩（条件列表）、条件项（条件取值）、动作桩（动作列表）、动作项（对应动作）。
- 适用：复杂的多条件判断逻辑（如"订单折扣"受"会员等级+订单金额+是否有优惠券"多个条件共同决定）

完整判定表实例（订单折扣规则：会员且金额>100享8折；会员且金额≤100享9折；非会员无折扣）：

条件桩	条件取值1	条件取值2	条件取值3	条件取值4
是否会员	Y	Y	N	N
金额>100	Y	N	Y	N
动作桩				
8折				
9折				
无折扣				

恍然大悟：判定表为什么能保证不漏组合？因为它把所有条件的所有取值组合穷举成表格——2个条件→4种组合，3个条件→8种组合，每种组合对应一个动作，一目了然。这是用"穷举组合"对抗"遗漏场景"，比人脑想"有哪些情况"靠谱得多。

5. 错误推测法

- 是什么：依靠测试人员的经验和直觉，推测程序可能出错的地方，针对性地设计测试用例的方法。不属于系统化方法，而是经验驱动的补充手段。
- 常见推测：空输入、超长输入、特殊字符（如引号、SQL注入字符）、零值、负数、边界附近的值
- 适用：作为其他方法的补充，捕捉系统方法可能遗漏的错误

6. 场景法

- 是什么：通过场景（业务流程）来设计测试用例的方法。场景由基本流（正常流程）和备选流（异常/分支流程）组成，覆盖用户实际使用软件时的各种路径。
- 适用：业务流程复杂的系统（如"网购"场景：登录→浏览→下单→支付→发货→收货，每一步都可能走备选流）
- 基本流：正常的、最常用的流程路径
- 备选流：异常的、备选的流程路径（如支付失败、库存不足）

易混辨析

考法	易错点
"黑盒测试依据什么"	依据需求规格说明书，不是源代码
"黑盒测试关注什么"	关注功能（外部行为），不关注内部逻辑
"边界值和等价类的关系"	边界值是等价类的补充，不是替代

记忆技巧

方法口诀："等边因判错场"（等价类、边界值、因果图、判定表、错误推测、场景法）

三、白盒测试 核心



核心概念

白盒测试（结构测试/基于结构的测试）：把程序看作一个"透明的盒子"，根据内部逻辑结构设计测试用例，检查程序内部的执行路径。

依据

- 程序流程图、源代码等内部设计文档

适用阶段

- 单元测试（关注代码逻辑）

白盒测试的逻辑覆盖法（必考排序）

逻辑覆盖法按覆盖强度从弱到强排列：

级别	名称	覆盖要求	强度
1（最弱）	语句覆盖	每条语句至少执行一次	最弱
2	判定覆盖（分支覆盖）	每个判定的真假分支至少执行一次	
3	条件覆盖	每个条件的真假至少执行一次	
4	判定/条件覆盖	同时满足判定覆盖和条件覆盖	
5	条件组合覆盖	每个判定中所有条件的各种组合都执行一次	
6（最强）	路径覆盖	所有可能路径至少执行一次	最强

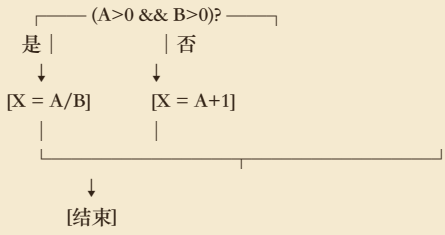
代码实例：用一段代码讲透6种覆盖（必看）

光看定义无法理解6种覆盖的差异。下面用一段含复合判定的代码，逐一演示每种覆盖需要多少测试用例。

被测代码：

```
if (A > 0 && B > 0)
    X = A / B;
else
    X = A + 1;
```

流程图（ASCII）：



这段代码有2个条件（A>0、B>0），1个判定（A>0 && B>0），2条路径（真分支、假分支）。

6种覆盖的测试用例对比：

覆盖标准	需要几个用例	具体取值示例	覆盖了什么
语句覆盖	2个	A=1,B=1（走真分支，执行X=A/B）；A=0,B=1（走假分支，执行X=A+1）	两条可执行语句都执行了



判定覆盖	2个	A=1,B=1 (真) ; A=0,B=1 (假)	判定的真假分支都走了
条件覆盖	2个	A=1,B=0 (A真B假) ; A=0,B=1 (A假B真)	A和B各自都取过真假, 但判定两次都为假!
判定/条件覆盖	2个	A=1,B=1 (真) ; A=0,B=0 (假)	既满足判定覆盖又满足条件覆盖
条件组合覆盖	4个	TT(A=1,B=1); TF(A=1,B=0); FT(A=0,B=1); FF(A=0,B=0)	4种组合全覆盖
路径覆盖	2个	A=1,B=1 (真分支) ; A=0,B=1 (假分支)	2条路径都走完

关键发现——条件覆盖 $\neq$ 判定覆盖的硬证据:

看条件覆盖那一行: 用例 A=1,B=0 和 A=0,B=1。

- A取过真(1)和假(0) 条件A覆盖
- B取过假(0)和真(1) 条件B覆盖
- 但判定 $A > 0 \ \&\& \ B > 0$ 两次都是假! (A真B假 $\rightarrow$ 假, A假B真 $\rightarrow$ 假)
- 真分支 ($X=A/B$) 从没执行过!

这就是"条件覆盖不一定包含判定覆盖"的硬证据——每个条件都覆盖了, 但整个判定的真分支没覆盖。

原因: 条件看的是"每个条件的局部真假", 判定看的是"整个判定的整体真假", 局部覆盖 $\neq$ 整体覆盖。

恍然大悟: 为什么条件覆盖和判定覆盖不可比? 因为它们看的"粒度"不同——条件看局部(每个条件), 判定看整体(整个判定)。局部都覆盖了, 整体可能没覆盖(如上例); 反过来整体覆盖了(判定真假都走), 局部也可能没覆盖(如只测A=1,B=1和A=0,B=0, B从未单独为真/假)。所以两者互不包含, 才需要"判定/条件覆盖"把它们结合起来。

基本路径测试法

- 基于McCabe环形复杂度
- 独立路径数 $= V(G) = E - N + 2$
- 至少需要 $V(G)$ 个测试用例覆盖所有独立路径

易混辨析 (重点中的重点)

考法	易错点
"语句覆盖和判定覆盖的关系"	判定覆盖一定包含语句覆盖 (实现判定覆盖就一定能实现语句覆盖)
"条件覆盖和判定覆盖的关系"	条件覆盖不一定包含判定覆盖! 两者不可比 (互不包含) ——这是高频坑点!
"路径覆盖最强是否就完美"	路径覆盖最强也不能保证发现所有错误 (含循环时无法穷举路径, 且路径覆盖不一定包含条件组合覆盖)
"白盒测试能发现什么"	白盒能发现逻辑错误 (死循环、路径未覆盖), 但看不到"功能错误" (用户想要的功能没实现)

覆盖率关系 (偏序关系, 非完全线性!)

这是本章最关键的考点。覆盖率的包含关系是偏序, 不是完全线性的:

- 语句覆盖 $\subseteq$ 判定覆盖 (判定覆盖一定包含语句覆盖)
- 条件覆盖 与 判定覆盖 不可比 (互不包含! 常考坑点!)
- 判定/条件覆盖 $\subseteq$ 判定覆盖 且 条件覆盖
- 条件组合覆盖 $\subseteq$ 判定/条件覆盖
- 路径覆盖是最强的覆盖标准, 但不一定包含条件组合覆盖 (路径关注"执行路径", 条件组合关注"条件取值组合", 两者维度不同, 即使路径全覆盖也不保证所有条件组合都取到)



强度排序（非严格包含链）：语句 < 判定 < 判定/条件 < 条件组合 < 路径

注意1：条件覆盖独立于这条链，它和判定覆盖不可比！

注意2：上述是"强度排序"，不是严格的"包含关系"——路径最强但不一定包含条件组合覆盖。

路径覆盖为什么不包含条件组合覆盖？（基于代码的硬证据）

用刚才的代码 `if (A>0 && B>0)` 说明：

- 路径只有2条：真分支（`X=A/B`）、假分支（`X=A+1`）
- 条件组合有4种：TT（A真B真）、TF（A真B假）、FT（A假B真）、FF（A假B假）
- 路径覆盖只需2个用例（走完2条路径），但这2个用例不可能覆盖4种组合——必然漏掉2种条件组合

合

比如用 `A=1,B=1`（TT，走真分支）和 `A=0,B=1`（FT，走假分支）做路径覆盖，2条路径都走完了，但 TF 和 FF 这2种条件组合从没试过！

恍然大悟：为什么最强也不够？因为路径和条件组合是两个维度——同一条路径可能对应多种条件组合（真分支只走TT，但TF/FT/FF都走假分支）。路径全覆盖只保证"每条路都走过"，不保证"每种条件组合都试过"。路径看"执行轨迹"，条件组合看"条件取值"，维度不同，所以路径覆盖不包含条件组合覆盖。

四、测试步骤（四个层次）

核心概念

软件测试按层次分四步：

1. 单元测试（模块测试）

- 测试对象：单个模块
- 测试方法：白盒为主
- 关注：模块内部逻辑
- 5项测试内容（简答题常考）：

1. 模块接口：测试参数个数/类型/顺序是否匹配（如调用时传了3个参数但定义只要2个）

2. 局部数据结构：测试模块内部变量是否正确（如变量未初始化、数组越界）

3. 边界条件：测试极端值（如数组下标0和n-1、循环0次和最大次数）

4. 独立执行路径：测试每条逻辑路径至少执行一次（对应语句/判定覆盖）

5. 错误处理路径：测试异常情况是否正确处理（如除零、文件不存在、网络断开）

恍然大悟：为什么单元测试测这5项而不是只测功能？因为这5项覆盖了模块可能出错的5个层面——接口错（传参不对）、数据错（内部变量）、边界错（极端值）、路径错（逻辑分支）、异常错（出错处理）。漏一项就漏一类bug，就像体检只查心脏不查肺，漏了哪个器官都可能致命。

2. 集成测试

- 测试对象：模块组合后的子系统/系统
 - 测试方法：白盒+黑盒结合
 - 集成策略（必考）：
 - 自顶向下：从主模块开始，逐层向下集成。需要桩模块（stub，模拟下层未实现的模块）
 - 自底向上：从底层模块开始，逐层向上集成。需要驱动模块（driver，模拟上层未实现的模块）
 - 三明治（混合）：自顶向下+自底向上结合
- 必背：自顶向下→桩模块；自底向上→驱动模块。方向相反，辅助模块也相反！

3. 系统测试

- 测试对象：整个系统
- 测试方法：黑盒为主



- 关注：系统整体功能是否符合需求

4. 验收测试

- 测试对象：交付前的系统
- 测试者：用户
- α测试与β测试（必考，以张海藩教材为准）

驱动模块与桩模块（高频坑点）

集成策略	需要的辅助模块	作用
自顶向下	桩模块（stub）	模拟下层未实现的模块（"假装下级存在"）
自底向上	驱动模块（driver）	模拟上层未实现的模块（"假装上级在调用"）

记忆技巧

口诀："顶桩底驱"——自顶向下用桩，自底向上用驱。谐音"顶住底气"：顶（自顶）住（桩）底（自底）气（驱）。

五、α测试与β测试 必考

核心概念（以张海藩教材为准）

测试	地点	测试者	开发者是否在场
α测试	开发者的环境	用户	通常不在场
β测试	用户的实际运行环境	用户	通常不在场

核心区分点

- 测试地点不同：α在开发环境，β在用户实际环境
- 测试阶段不同：α在前，β在后
- 开发者通常都不在场（不是区分点！）

易混辨析（最高频坑点）

常见错误说法	正确说法
"α测试开发者在场，β测试开发者不在场"	错！两者开发者通常都不在场。区分点是地点（开发环境vs用户环境），不是"开发者是否在场"
"α测试由开发者做"	错！α测试由用户做（在开发环境），不是开发者
"α测试和β测试都是开发者测试"	错！两者都是用户测试，只是地点不同

教材差异提示：张海藩教材中，α测试是"用户在开发者的环境中进行测试，开发者不在场"。部分工业界资料（如ISTQB）可能描述α测试"开发者可协助"，但期末考试以张海藩版为准：区分点是测试地点。

记忆技巧

口诀："α在开发地，β在用户地，都是用户测，开发者不在"。

类比：α像"去厂家门店试用"（在开发环境），β像"拿回家试用"（在用户实际环境）。



六、回归测试

核心概念

回归测试：在软件修改后（修复bug或增加功能后），重新运行之前的测试用例，确保修改没有引入新的错误、没有破坏原有功能。

为什么重要

- 修改一处代码可能"牵一发而动全身"
- 回归测试防止"改了一个bug，引入两个新bug"

回归测试深度拆解

- 是什么：对修改后的软件重新执行已有测试用例，验证修改没破坏已有功能
- 触发场景：

1. 修复bug后（验证修复+不引入新bug）
 2. 增加新功能后（验证新功能+不影响老功能）
 3. 重构代码后（验证行为不变）
 4. 修改配置/环境后（验证兼容性）
- 三种回归策略：

策略	选用范围	优点	缺点
全量回归	跑所有历史测试用例	最安全，覆盖最全	耗时长（大项目几小时甚至几天）
选择性回归	只跑受改动影响的相关用例	速度快	可能漏测受连锁影响的功能
优先级回归	按"风险×影响"排序，跑最重要的用例	平衡速度和覆盖	需维护用例优先级

- 自动化的必要性：

1. 手工回归不可行——每次改一行代码都跑1000个用例，人工无法承受
 2. 频繁触发——CI/CD要求每次提交都跑回归
 3. 降低边际成本——一次自动化投入，无限次复用
- 典型工具：Jenkins+JUnit/Selenium/PyTest 自动化测试套件

回归测试 vs 普通测试

维度	普通测试	回归测试
触发时机	新功能开发完成后	任何代码变更后
目的	发现新bug	防止旧功能被破坏
用例来源	新设计	复用历史用例库
执行频率	一次性	反复执行（每次变更都跑）

恍然大悟：回归测试是"修一个bug验证N个旧功能"——没有自动化就成了改一行代码测一周的瓶颈。这就是为什么大型软件公司必须建CI/CD流水线——不是为了"先进"，而是因为没有自动化回归，软件就只能停在"小作坊"规模。回归测试的价值不在"找bug"，而在"防退化"——它是软件迭代的"刹车"，没有它就不敢改代码。这也是为什么TDD（测试驱动开发）被推崇——先写测试就是为了让回归测试集自然累积，越改代码越安全。

记忆技巧

口诀："改后回归，防新错"——修改后回归测试，防止引入新错误。

七、调试 (Debug)

核心概念

调试是测试发现错误后，定位并修复错误的过程。

调试方法

1. 强行排错：通过输出调试信息、设置断点等方式硬找
2. 回溯法：从发现错误的地方逆推，追溯错误源头
3. 原因排除法：排除不可能的原因，缩小范围

三种调试方法深度拆解

1. 强行排错 (Brute Force)

- 是什么：通过大量打印调试信息、断点、内存转储等手段"硬找"错误
- 常用手段：
 - 在可疑位置插入大量print/log语句
 - 用调试器设置断点逐行查看变量值
 - 内存/堆栈转储 (dump) 分析崩溃现场
 - 全程跟踪日志重现问题
- 优点：直接简单（不需要太多思考）；适合不熟悉系统的新人
- 缺点：

1. 盲目低效——撒大网捞鱼，输出大量无关信息
2. 生产环境不友好——日志泛滥影响性能
3. 不培养调试思维——只会按方法用工具，不会推理
 - 适用：错误位置完全不明、其他方法无从下手时的"最后手段"

2. 回溯法 (Backtracking)

- 是什么：从发现错误的现象出发，沿着控制流逆向追溯到错误源头
- 机制：

1. 定位错误现象（哪行代码出错、什么报错信息）
2. 沿调用栈/控制流逆向回溯（这行代码怎么执行到的？输入哪来？）
3. 逐层向上检查，直到找到错误根因
 - 优点：思路清晰、有方向（不撒大网）；适合定位明确的错误
 - 缺点：

1. 程序大时分支爆炸——回溯路径太多无法穷举
2. 依赖调用栈完整性——异步/事件驱动场景调用栈断裂时失效
 - 适用：小程序或错误位置已明确（如某行抛异常）；过程式代码
3. 原因排除法 (Cause Elimination)

- 是什么：通过提出假设、排除假设的方式，逐步缩小错误范围
- 三种子方法：

1. 演绎法 (Deduction)：列出所有可能原因 → 收集数据排除不可能 → 剩下的就是真凶



- 类比：福尔摩斯式破案（"排除一切不可能后，剩下的不论多么不可信都是真相"）
2. 归纳法（Induction）：从多个出错现象中找共同点 → 总结规律推断根因
 - 类比：从多次崩溃日志中发现"都发生在凌晨3点" → 推断是定时任务冲突
 3. 二分定位法（Binary Search）：把代码/数据折半，定位错误在哪一半 → 再折半
 - 类比：git bisect 二分查找引入bug的提交
 - 优点：思路系统化；适用范围广；培养工程思维
 - 缺点：需要一定经验和数据；前期分析耗时
 - 适用：复杂bug、间歇性bug、生产环境bug

三种方法选用对比

方法	适用场景	思维模式
强行排错	完全无头绪、新人	撒大网（蛮力）
回溯法	小程序、错误明确	追踪嫌疑人（向后追）
原因排除法	复杂系统、间歇bug	排除嫌疑（推理）

恍然大悟：调试是"破案"，三种方法对应三种破案思路——强行排错=撒大网（嫌疑人不明就广撒网），回溯法=追踪嫌疑人（已知现场往回追），原因排除法=排除嫌疑（系统化推理）。新手往往只会强行排错（打印满屏log），高手能快速选对方法——简单错用回溯法（5分钟解决），复杂错用原因排除法（避免迷路）。调试能力的提升不是工具熟练度，而是方法选择智慧。这也是为什么有经验的工程师调试快——不是手快，是脑子选方法快。

记忆技巧

口诀："强回原"（强行排错、回溯法、原因排除法）。
方法选择口诀："无头绪强行，错明确回溯，复杂用排除"。

八、软件维护

核心概念

软件维护是软件交付使用后，为了改正错误或满足新的需求而修改软件的过程。

四种维护类型（必背+占比）

类型	目的	占比
改正性维护	修改交付后发现的错误	~20%
适应性维护	适应环境变化（新操作系统、新硬件）	~25%
完善性维护	增加新功能、改善性能（用户要求改进）	~50%以上（最大！）
预防性维护	预防未来可能的问题（主动改进）	~5%

易混辨析

考法	易错点
"哪种维护占比最大"	完善性维护（50%以上），不是改正性维护
"适应性vs完善性"	适应性=适应环境变化（被动）；完善性=增加新功能/改善性能（主动改进）
"改正性vs预防性"	改正性=事后修bug；预防性=事前预防



软件可维护性

软件可维护性由三大特性决定，每个特性都有对应的度量和设计实践：

1. 可理解性 (Understandability)

- 是什么：维护人员能否快速读懂代码意图
- 度量指标：
 - 圈复杂度 (McCabe V(G))：>10 的模块难理解
 - 注释比例：合理 20-30% (过多冗余、过少难懂)
 - 命名规范度：变量/函数名能否表达意图
- 设计实践：
 - 高内聚 (一个模块只做一件事，容易理解)
 - 命名清晰 (避免 `a`、`temp`、`data1` 等)
 - 注释解释 "为什么" 而非 "做什么"
 - 反例：函数名 `process()`，10 个参数，100 行代码 → 三个月后作者自己都看不懂

2. 可测试性 (Testability)

- 是什么：能否方便地为代码编写测试用例
- 度量指标：
 - 单元测试覆盖率 (>80% 为良好)
 - 测试用例数与代码行的比例
 - 是否有外部依赖 (依赖越多越难测)
- 设计实践：
 - 低耦合 (依赖少则易 mock)
 - 依赖注入 (外部依赖通过参数传入，便于替换为 mock)
 - 单一职责 (一个函数只做一件事，便于针对性测试)
 - 反例：函数直接调用数据库/网络/全局变量 → 写单元测试要先搭整套环境

3. 可修改性 (Modifiability)

- 是什么：修改一处不影响其他地方的能力
- 度量指标：
 - 模块间耦合度
 - 圈复杂度
 - 修改一个需求需要改的文件数 (<5 为良好)
- 设计实践：
 - 信息隐藏 (封装可变的设计决策)
 - 开闭原则 (对扩展开放、对修改关闭)
 - DRY 原则 (同一逻辑不重复出现)
 - 反例：业务规则硬编码散落在 20 个文件 → 改个税率要找一周

三特性的内在关系

可维护性 = 可理解性 × 可测试性 × 可修改性 (任何一个为 0 维护性都为 0)

特性	解决的问题	设计阶段对应原则
可理解性	"看不懂"	第四章高内聚、命名规范
可测试性	"测不动"	第四章低耦合、依赖注入
可修改性	"改不动"	第四章信息隐藏、开闭原则

恍然大悟：可维护性是设计阶段决定的——代码写下去那一刻就锁定了维护成本，事后无法补救。这就是为什么前面第四章反复强调高内聚低耦合、信息隐藏、开闭原则——这些都不是 "为了好看"，而是为了降低未来 5-10 年的维护成本。可维护性差的软件不是 "维护时多花点时间"，而是到一定程度后无法维护，只能推翻重写——这是大型软件最贵的代价。可维护性的 "事后补救" 成本是 "事前设计" 的

10-100倍——这也是为什么资深工程师宁可设计阶段多花一周，也不愿事后维护多花一年。

维护成本

- 软件维护成本远高于开发成本（维护期占生命周期的大部分）

软件维护的副作用（考点）

维护是"在运行的系统上动刀"，修改可能引入三类副作用：

副作用类型	含义	具体场景例子
代码副作用	修改代码引入新的错误	改支付逻辑时误删了一行校验，导致用户超支
数据副作用	修改数据结构破坏现有数据	把"金额"字段从int改成float，历史数据精度丢失
文档副作用	改了代码没改文档，导致不一致	改了接口参数但没更新API文档，调用方报错

恍然大悟：为什么维护比开发还难？因为维护是"在运行的系统上动刀"——改代码可能引入新bug（代码副作用），改数据结构可能破坏现有数据（数据副作用），改了代码忘了改文档导致后续维护者被误导（文档副作用）。三重风险叠加，所以维护成本远高于开发。这就是为什么前面强调"高内聚低耦合""信息隐藏"——好的设计能降低维护时牵一发动全身的风险。

记忆技巧

类型口诀："改适完预"（改正性、适应性、完善性、预防性）。

占比口诀："完善过半，改适各四分之一，预防最少"——完善性50%以上，改正性和适应性各约20-25%，预防性最少。

考点聚焦

高频考点清单

1. 白盒逻辑覆盖排序（语判条判组路，偏序关系）
2. 条件覆盖与判定覆盖不可比（高频坑点！）
3. α 测试vs β 测试（区分点是地点，非开发者是否在场）
4. 集成测试策略：自顶向下→桩模块，自底向上→驱动模块
5. 维护四种类型及占比（完善性最大）
6. 黑盒测试6种方法
7. 边界值是等价类的补充（非替代）
8. 测试目标是发现错误（非证明正确）
9. 路径覆盖最强也不能保证完全正确
10. McCabe复杂度=独立路径数=最少测试用例数

常见题型

- 排序题：逻辑覆盖从弱到强
- 判断题：条件覆盖是否包含判定覆盖（不包含！不可比！）
- 场景题： α/β 测试判断、维护类型判断
- 计算题：McCabe复杂度→独立路径数



记忆口诀总览

1. 测试目标: "测试找错, 不证对"
 2. 黑盒方法: "等边因判错场"
 3. 白盒覆盖: "语判条判组路" (从弱到强)
 4. 覆盖关系: "判定包语句, 条件不可比, 组合包判定/条件, 路径最强不包组合"
 5. 集成策略: "顶桩底驱" (顶住底气)
 6. α/β 测试: " α 在开发地, β 在用户地, 都是用户测, 开发者不在"
 7. 回归测试: "改后回归, 防新错"
 8. 调试方法: "强回原"
 9. 维护类型: "改适完预"
 10. 维护占比: "完善过半, 改适各四分之一, 预防最少"
-

本章小结

本章是全书最重要的考点章。核心逻辑链: 测试目标 (发现错误) $\rightarrow$ 黑盒 (看功能) + 白盒 (看逻辑) $\rightarrow$ 分层测试 (单元 $\rightarrow$ 集成 $\rightarrow$ 系统 $\rightarrow$ 验收) $\rightarrow$ 维护 (改适完预)。

本章最大的坑点集中在三处:

1. 覆盖率的偏序关系: 条件覆盖和判定覆盖不可比 (不是简单的线性包含), 这是出题人最爱设的陷阱
2. α/β 测试的区分: 区分点是测试地点 (开发环境vs用户环境), 不是"开发者是否在场"——两者开发者通常都不在场
3. 集成测试的辅助模块: 自顶向下用桩模块, 自底向上用驱动模块, 方向相反辅助模块也相反
掌握这三个坑点, 加上覆盖率排序和维护类型占比, 本章的分数就能稳稳拿下。

